Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Dersin tarihi 8 Kasım 2021 Pazartesi. Ders çevrim içi yapıldı.
  - Asıl anlatım yaklaşık 00:06'da başlar. Öncesi bekleme ve önceki dersten kalan `fork` sorusunun cevabıdır.
  - Hocanın kamerası bu derste çalışmadı. Sadece ekran paylaşımı ve ses var.
  - Bu dersle Bölüm 3 (süreçler) bitti. Sonraki derste iş parçacıklarına (threads) geçilecek.
- Önceki dersin sorusu: `fork` sonrası ebeveyn ve çocukta farklı olan iki bilgi.
- Süreçler arası iletişim (IPC)
  - Paylaşımlı bellek ve mesaj geçirme modelleri, karşılaştırma.
  - Bağımsız ve işbirlikçi süreçler.
  - Üretici-tüketici (producer-consumer) problemi, sınırsız (unbounded) ve sınırlı (bounded) tampon (buffer), dairesel tampon kodu.
- Mesaj geçirme sisteminin tasarım konuları
  - Sabit ya da değişken mesaj boyutu.
  - Doğrudan (direct) ve dolaylı (indirect) iletişim, posta kutusu (mailbox) / port.
  - Tek yönlü (unidirectional) ve çift yönlü (bidirectional) bağlantı.
  - Engelleyen (blocking) ve engellemeyen (non-blocking) gönderme/alma, senkron ve asenkron.
  - Tamponlama (buffering): sıfır, sınırlı ve sınırsız kapasite.
- IPC sistemi örnekleri
  - POSIX paylaşımlı bellek (`shm_open`, `ftruncate`, `mmap`, `shm_unlink`), üretici ve tüketici kodu.
  - Mach'ta mesaj geçirme (portlar).
  - Windows ALPC (Advanced Local Procedure Call).
- İstemci-sunucu (client-server) iletişimi
  - Soketler (sockets), Java ile tarih sunucusu (date server) örneği.
  - Uzak yordam çağrısı (RPC, Remote Procedure Call), koçanlar (stubs), sıralama (marshalling).
  - Sıradan borular (ordinary pipes) ve isimli borular (named pipes / FIFO).

Hocanın Özellikle Vurguladığı Kısımlar

- `fork` sonrası ebeveyn ve çocuk süreç kontrol blokları neredeyse aynıdır, ama iki bilgi farklıdır.
  - Süreç kimliği (PID): her süreç için tektir.
  - Ebeveyn süreç kimliği: çocuğun ebeveyni, onu yaratan süreçtir.
- Paylaşımlı bellekte senkronizasyon mekanizmasını işletim sistemi sunar, ama senkronizasyonu sağlamak programcının sorumluluğudur.
  - Kim, alanın neresine yazdı; diğeri nereden okuyacak; üzerine yazılacak mı? Bunların hepsi programcının kontrolündedir.
- Mesaj geçirmede en az iki kopyalama vardır: gönderenden çekirdeğe, çekirdekten alıcıya.
  - Bu yüzden büyük veri bloklarında verimsizdir; genelde küçük veriler için kullanılır.
- Tek yönlü bir yapı, iki tane kullanılarak çift yönlü hâle getirilebilir.
  - Örnek: zıt yönlerde iki boru.
- Hem gönderme hem alma engelleyen (blocking) olursa randevu (rendezvous) oluşur.

  - Üretici-tüketici problemi çok basitleşir, ama iki taraf birbirinin hızına bağlanır; verimli değildir.
- POSIX paylaşımlı bellek ve diğer IPC yöntemleri uygulama saatlerinde ayrıca gösterilecek. POSIX mesaj kuyrukları (message queues) büyük ihtimalle 3. uygulamada.
- **Uygulamalar (laboratuvar):**
  - Uygulama büyük ihtimalle ertesi gün (9 Kasım) yapılacak.
  - Uygulama bir izleme ve öğrenme ortamı. Gösterilenleri kendiniz de deneyin.
  - Her şey Linux üzerinde gösterilecek. Elinizin altında Linux kurulu bir makine ya da sanal makine (virtual machine) olsun.
  - Bilgisayarınız yeterliyse (ör. 8-16 GB RAM) sanal makine esneklik sağlar. Hoca, donanımı bilmeden kesin öneri vermenin doğru olmadığını söyledi.
  - Uygulamada anlatılanlardan sorumlusunuz. Vize ve finalde hem teorisi hem pratiği sorulabilir.
  - Uygulamaları üç asistan yürütüyor.

Kısa Tekrar Notları

- `fork` sonrası farklı olanlar: PID ve ebeveyn PID'si.
- Paylaşımlı bellek: aynı fiziksel bellek alanı iki sürecin adres uzayına eşlenir. Genelde aynı makinedeki süreçler için. Dağıtık sistemde de mümkün (ör. InfiniBand) ama pahalı.
- Mesaj geçirme: çekirdekte bir mesaj kuyruğu, posta kutusu ya da port. Dağıtık sistemlerde tercih edilir.
- POSIX her iki yöntemi de tanımlar.
- Sınırlı tampon (boyut `BUFFER_SIZE`): `in == out` ise tampon boştur; `(in + 1) % BUFFER_SIZE == out` ise doludur. En fazla `BUFFER_SIZE - 1` eleman tutulabilir.
- Mesaj boyutu sabitse işletim sisteminin işi kolay, programcının işi zor. Değişkense tersi.
- Doğrudan iletişim: `send(P, message)`, `receive(Q, message)`. Bağlantı otomatik kurulur; her çift arasında tam olarak bir bağlantı vardır.
- Dolaylı iletişim: `send(A, message)`, `receive(A, message)` (A bir posta kutusu). Bir bağlantı ikiden fazla süreçle ilişkili olabilir. İki süreç arasında birden fazla bağlantı olabilir.
- Engelleyen = senkron. Engellemeyen = asenkron.
- Tamponlama: sıfır kapasite (gönderen alıcıyı bekler), sınırlı kapasite (doluysa gönderen bekler), sınırsız kapasite (gönderen hiç beklemez).
- POSIX paylaşımlı bellek: `shm_open` → `ftruncate` → `mmap` → yaz/oku → `shm_unlink`.
- Mach: her şey mesajla; her görev (task) yaratılınca Kernel ve Notify posta kutularını alır.
- Windows ALPC: bağlantı portu (connection port) ve iletişim portu (communication port).
- Soket = IP adresi + port numarası. 1024'ün altındaki portlar iyi bilinen (well-known) portlardır.
- RPC: uzak sistemdeki bir fonksiyonu yerel çağrı gibi çağırma. Koçanlar parametreleri sıralar (marshalling).
- Sıradan boru: yalnızca ebeveyn-çocuk arasında, tek yönlü. `fd[0]` okuma ucu, `fd[1]` yazma ucu.
- İsimli boru (FIFO): `mkfifo` ile dosya sisteminde bir girdi olarak yaratılır; adını bilen her süreç kullanabilir.

Detaylı Açıklamalar

Önceki dersin sorusu

`fork` sonrası iki sürecin PCB'leri teknik olarak aynıdır, ama iki bilgi farklı olmalıdır:

1. **PID:** Her sürecin ayrı bir kimliği olmalıdır.
2. **Ebeveyn PID'si:** Her süreç başka bir süreç tarafından yaratılır. Çocuğun ebeveyni, onu yaratan süreçtir. Bir süreç birden fazla çocuk yaratabileceği için bu karşılaştırma ebeveyn-çocuk çifti içindir, sistemdeki bütün süreçler için değil.

Paylaşımlı bellek ve mesaj geçirme

Paylaşımlı bellek: İki süreç aynı fiziksel bellek bölgesine erişir.

- Normalde işletim sistemi süreçlerin adres uzaylarını birbirinden yalıtır. Paylaşımlı bellek bu kuralın istisnasıdır; iki süreç de buna onay verir.
- Mesaj bir kez yazılır, diğer süreç oradan okur. Kopyalama yoktur.
- Genelde aynı makinedeki süreçler için kullanılır.

Mesaj geçirme: Ortak bellek yerine çoğunlukla çekirdekte bir mesaj kutusu ya da port yaratılır. Gönderen mesajı buraya koyar, alıcı buradan alır.

- Kuyruk çekirdekteyse her işlem için sistem çağrısı gerekir.
- Mesaj en az iki kez kopyalanır: A'dan çekirdeğe, çekirdekten B'ye.
- Bu yüzden genelde küçük veriler için kullanılır.

İki yöntemde de senkronizasyon gerekir: mesaj yazıldı mı, okundu mu, sonraki mesaj nereye yazılacak?

Bağımsız ve işbirlikçi süreçler

- **Bağımsız süreç:** diğer süreçlerle etkileşmez, onların çalışmasından etkilenmez.
- **İşbirlikçi süreç:** diğer süreçlerle haberleşir; aralarında bağımlılık vardır. Uyumu bozan bir hareket hepsini etkiler.

İşbirliği neden istenir?

- Bilgi paylaşımı.
- Hesaplamayı hızlandırma.
- Modülerlik. Örnek işleme hattı: bir süreç kameradan görüntü alır, biri ön işleme yapar, biri yüz tespiti (face detection) yapar.
- Kullanım kolaylığı.

Üretici-tüketici problemi

Bilgisayar sistemlerinde (işletim sistemleri, ağlar vb.) birçok iş bu probleme çevrilebilir. Örnek: istemci-sunucu mimarisinde sunucu bilgiyi üretir, istemci tüketir (ya da tersi).

- **Sınırsız tampon:** Üretici hiç beklemez. Teoride mümkün, pratikte yok.
- **Sınırlı tampon:** Tampon doluysa üretici bekler. Tampon boşsa tüketici bekler (bu iki durumda da geçerli).

Slayttaki paylaşımlı veri:

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Üretici:

```
item next_produced;
while (true) {
    /* produce an item in next_produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Tüketici:

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
```

```
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in next_consumed */
}
```

- `in`: sıradaki elemanın yazılacağı göz. `out`: sıradaki elemanın okunacağı göz.
- Tampon dairesel (circular) olduğu için indeksler `% BUFFER_SIZE` ile başa döner.
- Başta `in == out == 0` ve 0. gözde bir şey yoktur. Yani `in == out` tamponun **boş** olduğunu gösterir.
- Tamponun dolu olduğu, `in`'in bir adım sonrası `out`'a eşitse anlaşılır.
- Bu yüzden 10 gözlük tamponda en fazla 9 eleman tutulabilir. Slayt da bunu söyler: "Solution is correct, but can only use `BUFFER_SIZE-1` elements". Daha iyi bir çözüm için ayrıca bir sayaç ya da senkronizasyon aracı gerekir.

Mesaj geçirme sisteminin tasarımı

Mesaj geçirme sisteminin sunması gereken iki temel işlem `send(message)` ve `receive(message)`'dır. Ek parametreler şu kararlara göre değişir: iletişim doğrudan mı dolaylı mı, senkron mu asenkron mu, tamponlama nasıl?

Mesaj boyutu:

	Sabit boyut	Değişken boyut
İşletim sistemi	Tampon yönetimi kolay (ör. 10 mesaj × 1 KB)	Her mesajın önüne boyutu belirten bir başlık (header) gerekir
Programcı	Büyük mesajları parçalayıp karşıda birleştirmek gerekir	İhtiyaca göre 500 B ya da 1500 B gönderilebilir

Doğrudan iletişim: Süreçler birbirini açıkça adlandırır.

- `send(P, message)`: P sürecine mesaj gönder.
- `receive(Q, message)`: Q sürecinden mesaj al.
- Bağlantı otomatik kurulur ve yalnızca bu iki süreç arasındadır.
- Her süreç çifti arasında tam olarak bir bağlantı vardır.
- Bağlantı tek yönlü olabilir, genelde çift yönlüdür.

Dolaylı iletişim: Mesajlar posta kutusuna (mailbox, port) gönderilir ve oradan alınır. Gerçek hayattaki posta kutusuna benzer.

- `send(A, message)`, `receive(A, message)`: A bir posta kutusu.
- Bağlantı yalnızca posta kutusunu paylaşan süreçler arasında kurulur.
- Bir bağlantı ikiden fazla süreçle ilişkili olabilir.
- İki süreç arasında birden fazla bağlantı (birden fazla posta kutusu) olabilir.
- Gerekli işlemler: posta kutusu yaratma, gönderme/alma, posta kutusunu yok etme.
- Doğrudan iletişimdeki simetri yoktur; herkes aynı posta kutusunu adresler.

Posta kutusu paylaşımı sorunu: P1, P2 ve P3 aynı posta kutusunu paylaşsın. P1 mesaj gönderir. Mesajı P2 mi P3 mü alır? Olası çözümler:

- Bir bağlantının en fazla iki süreçle ilişkili olmasına izin vermek.
- Aynı anda yalnızca bir sürecin `receive` yapmasına izin vermek.
- Alıcıyı sistemin seçmesine izin vermek (ör. sıra ya da öncelikle). Sistem, mesajın kime verildiğini gönderene bildirir.

Engelleme (blocking):

	Engelleyen (senkron)	Engellemeyen (asenkron)
send	Gönderen, mesaj alınana kadar bekler	Gönderen mesajı gönderip devam eder

	Engelleyen (senkron)	Engellemeyen (asenkron)
receive	Alıcı, mesaj gelene kadar bekler	Alıcı geçerli bir mesaj ya da null alır, devam eder

Hocanın örneği: WhatsApp'ta her mesajın okunduğunu (mavi tik) görmeden ikincisini göndermemek engelleyen göndermeye benzer. Telefonun başında mesaj gelene kadar beklemek engelleyen almaya benzer.

Gönderme ve alma farklı biçimlerde birleştirilebilir. İkisi de engelleyense randevu (rendezvous) oluşur. Üretici `send` der ve tüketici alana kadar bekler; tüketici `receive` der ve mesaj gelene kadar bekler. Ek kontrol gerekmez, ama iki süreç farklı hızlarda çalışamaz.

Tamponlama: Bağlantıya bağlı bir mesaj kuyruğu vardır. Kapasitesi:

- **Sıfır:** Kuyrukta mesaj bekleyemez. Gönderen alıcıyı beklemek zorundadır (randevu).
- **Sınırlı (n mesaj):** Kuyrukta yer varsa gönderme engellemesizdir. Kuyruk doluysa gönderen yer açılana kadar bekler.
- **Sınırsız:** Gönderen hiç beklemez. Gerçekte yoktur.

POSIX paylaşımlı bellek

Paylaşımlı bellek, belleğe eşlenen bir dosya (memory-mapped file) gibi kullanılır. İşlemler dosya işlemlerine benzer.

1. `shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`
   - Nesneye bir ad verilir. `O_CREAT`: yoksa yarat. `O_RDWR`: okuma ve yazma. `0666`: izinler; hangi süreçlerin erişebileceğini sınırlar.
   - Dönen değer, alt seviye `open` gibi bir dosya tanımlayıcısıdır (file descriptor).
2. `ftruncate(shm_fd, 4096);` nesnenin boyutunu 4096 bayt yapar.
3. `ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);` nesneyi belleğe eşler. `ptr` artık bu alanın adresidir.
4. Üretici iki mesaj yazar. Her yazmadan sonra `ptr`'yi mesajın uzunluğu (`strlen`) kadar ilerletir, böylece ikinci mesaj birincinin üzerine yazılmaz.
5. Tüketici aynı adla `shm_open(name, O_RDONLY, 0666)` yapar; yalnızca okuyacağı için boyut ayarlamaz. Eşledikten sonra iki parça hâlinde yazılmış mesajı tek seferde okur.
6. İş bitince `shm_unlink(name)` ile nesneyi kaldırır.

Slayttaki örnek kodu kendi denememde çalıştırdım: üreticinin "Hello" ve "World!" diye iki parça yazdığı alanı tüketici "HelloWorld!" olarak tek seferde okudu. Örnek yalnızca yapıların kullanımını göstermek içindir.

Mach'ta mesaj geçirme

Mach bir mikroçekirdektir. Sistem çağrıları bile mesajla yapılır.

- Posta kutularına Mach'ta port denir.
- Her görev (task) yaratılınca iki posta kutusu alır: **Kernel** (çekirdekle iletişim) ve **Notify** (olay bildirimleri).
- Mesaj aktarımı için üç sistem çağrısı vardır: `msg_send()`, `msg_receive()`, `msg_rpc()`.
- Yeni posta kutusu `port_allocate()` ile yaratılır.
- Aynı göndericiden gelen mesajlar FIFO sırasıyla kuyruğa girer. Farklı göndericiler arasındaki sıra garanti edilmez. Örnek: A1, B1, A2, B2 sırası garanti değil, ama A1 her zaman A2'den, B1 her zaman B2'den önce gelir.
- Mesajlar değişken boyutlu olabilir; önlerinde boyutu belirten bir başlık vardır.

Posta kutusu doluysa gönderen dört şey yapabilir:

1. Kuyrukta yer açılana kadar süresiz beklemek.
2. En fazla n milisaniye beklemek (zaman aşımı, timeout). POSIX mesaj kuyruklarında da bu seçenek vardır.
3. Hemen geri dönmek.

4. Mesajı geçici olarak çekirdeğe bırakmak (önbelleğe almak). Kuyrukta yer açılınca çekirdek mesajı yerleştirir ve gönderene bildirim gönderir.

Ayrıca birden fazla posta kutusu bir posta kutusu kümesi (port set) olarak birleştirilebilir. `receive` bu kümedeki herhangi bir posta kutusundan ya da belirli birinden okuyabilir. Mesaj yoksa ya hemen devam edilir ya da belli bir süre beklenir.

Windows ALPC

Windows'taki mesaj geçirme yapısının adı ALPC'dir (Advanced Local Procedure Call). Yalnızca aynı makinedeki süreçler arasında çalışır.

1. Sunucu süreç, herkesin görebileceği bir **bağlantı portu (connection port)** yayımlar.
2. İstemci bu porta bağlantı isteği gönderir.
3. Sunucu bir kanal ve bir çift **iletişim portu (communication port)** yaratır: biri istemciden sunucuya, biri sunucudan istemciye.

Mesaj boyutuna göre üç yöntem vardır:

- **Küçük mesajlar (256 bayta kadar):** portun mesaj kuyruğu ara bellek olarak kullanılır; mesaj bir süreçten diğerine kopyalanır.
- **Daha büyük mesajlar:** kanalla ilişkili paylaşımlı bir bellek bölgesi (section object) kullanılır.
- **Section object'e sığmayacak kadar büyük veriler:** sunucunun istemcinin adres alanını doğrudan okuyup yazmasını sağlayan bir API kullanılır.

Soketler

Soket, iletişim için tanımlanmış bir uç noktadır (endpoint). TCP/IP'de iletişim için iki adres gerekir:

- **IP adresi:** hedef bilgisayarı belirtir (ör. 161.25.19.8).
- **Port numarası:** o bilgisayardaki süreci belirtir. Port bilgisini taşıma katmanı (transport layer) sağlar.

Soket, iki uçtaki IP ve port bilgisini (çoğu zaman kullanılan protokol bilgisini de) bir arada tutar.

- 1024'ün altındaki portlar iyi bilinen portlardır ve IANA tarafından standart hizmetlere ayrılmıştır. Örnekler: 21 FTP, 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS.
- İstemcinin port numarası önemli değildir; sistemdeki boş bir porttan bağlanır.
- Sunucunun port numarası sabit olmalıdır ki ona her zaman ulaşılabilsin.

Java tarih sunucusu örneği: Sunucu 6013 numaralı portta dinler. Sonsuz döngüde her bağlantıyı kabul eder, sistem tarihini sokete yazar, bağlantıyı kapatır. Sunucu çok süreçli ya da çok iş parçacıklı değildir; istekleri sırayla işler. İstemci kodu slaytta yok. İstemci `new Socket("127.0.0.1", 6013)` gibi bir soket açar ve tarihi okur.

Uzak yordam çağrısı (RPC)

Amaç, yerel bir fonksiyon çağrısını dağıtık bir ortamda da aynı kolaylıkla yapmaktır.

- İki sistem farklı olabilir: biri little-endian, diğeri big-endian olabilir; tamsayı boyutları farklı olabilir.
- Bu farkları **koçanlar (stubs)** giderir. Koçan parametreleri ağdan gönderilebilecek biçime sokar. Buna sıralama (marshalling) denir.
- Windows'ta koçan kodu, MIDL (Microsoft Interface Definition Language) ile yazılmış bir tanımdan derlenir.

Borular

Sıradan borular (ordinary pipes): IPC'nin en eski ve en basit yöntemlerinden biridir.

- Yalnızca ebeveyn-çocuk ilişkisi olan süreçler arasında kullanılır.
- `pipe()` çağrısı iki tanımlayıcı döndürür: biri borunun okuma ucu, biri yazma ucu.

- Boru yaratıldıktan sonra `fork` yapılırsa çocuk da aynı boruya erişir.
- İletişim tek yönlüdür. Ebeveyn yazacaksa okuma ucunu kapatır; çocuk okuyacaksa yazma ucunu kapatır.
- Zıt yönlerde iki boru kullanılarak çift yönlü iletişim kurulabilir.

İsimli borular (named pipes, FIFO):

- `mkfifo` ile yaratılır. Dosya sisteminde bir girdi olarak görünür.
- Adını bilen her süreç kullanabilir; ebeveyn-çocuk ilişkisi gerekmez.
- Ayrıntılar uygulama saatinde gösterilecek.

Ders sonrası notlar

- **Borunun uçları:** Hoca "ebeveyn yazma ucuna sıfırdan, okuma ucuna birden erişiyor" dedi. Doğrusu tersidir: `fd[0]` **okuma** ucu, `fd[1]` **yazma** ucudur. Küçük bir C programıyla denedim: çocuk `fd[1]`'e "hi" yazdı, ebeveyn `fd[0]`'dan okudu.
- **Port sayısı:** Derste "1024 yani 2 üzeri 16 tane portumuz var" dendi. TCP ve UDP'de port numarası 16 bittir; toplam $2^{16} = 65536$ port vardır (0-65535). İyi bilinen portlar bunların ilk 1024'üdür (0-1023).
- **Mach'ta önbelleğe alınan mesaj sayısı:** Derste "posta kutusu başına bir" dendi. Kitaptaki kural büyük olasılıkla, her gönderici iş parçacığı için dolu bir posta kutusunda aynı anda yalnızca bir mesajın bekleyebileceğidir.
- **Windows ALPC:** Derste section object'in boyut sınırının "daha sınırlı" olduğu söylendi. Section object, 256 bayttan büyük mesajlar içindir. Ondan da büyük veriler için doğrudan adres alanı okuma/yazma API'si kullanılır. Section object'in kesin bir boyut sınırı derste verilmedi.
- **İsimli boruların yönü:** Altyazıda bu kısım belirsiz. UNIX'te FIFO'lar büyük olasılıkla çift yönlü kullanılabilir ama yarı çift yönlüdür (half-duplex): aynı anda tek yönde veri akar. İki yönlü eşzamanlı iletişim için genelde iki FIFO kullanılır.
- **Soketler:** Altyazıda "soketleri sadece internet ortamında kullanmak zorundasınız" diye geçen cümle anlam olarak tersidir. Hoca farklı soket türleri olduğunu söylüyor. Örneğin aynı makinedeki süreçler için UNIX domain soketleri vardır.
- **Sınırlı tampon:** Slayttaki çözümde 10 gözlük tamponun en fazla 9 elemanla dolduğunu Python ile denedim.
- **Port örnekleri:** Altyazıda "21-22" diye geçen yerde 21 FTP, 22 SSH'tır.
- **Altyazı hataları:** "Mac" = Mach; "PostX" ve "COSIX" = POSIX; "LPC/ALPC" = Advanced Local Procedure Call; "dpc channel" = ALPC kanalı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.